CSA0312 – Data Structures (Slot B) – Final Assignment Report

Intelligent Emergency Evacuation Routing System

Field	Value
Course	CSA0312 – Data Structures – Slot B
Assignment	Intelligent Emergency Evacuation Routing System for a city with more than 8,000 intersections and 20,000 weighted roads
Student name	_____
Register number	_____
Team members / contributions	_____
Date of submission	_____
GitHub Repository	[Insert repository link after upload]

> This project is a C prototype evaluated on deterministic synthetic data.
> No real-world deployment or real disaster validation is claimed.

---

1. Problem Understanding

A smart-city disaster management authority needs a routing service that guides evacuees from disaster locations to evacuation centers. The road network is large (more than 8,000 intersections, more than 20,000 weighted

roads) and, during a disaster, it is unstable: congestion and damage change road costs continuously, and some roads become impassable.

The computational problem is therefore **single-source, single-target shortest path on a large, sparse, dynamically changing, non-negatively weighted graph**, with these operational demands:

- many independent source-to-destination queries;
- the *same* query repeated many times as evacuees and dispatchers re-ask for directions, which motivates a route cache;
- road-weight updates and road blocking/reopening between queries, which means cached answers can silently become wrong and must be invalidated;
- unreachable evacuation centers must be *reported*, not treated as an error or an infinite loop;
- the network contains cycles, so the algorithm must never process the same intersection twice;
- moderate memory use — an $O(V^2)$ structure is already ~67 million cells for V = 8,200, which rules out an adjacency matrix and Floyd–Warshall tables at full scale.

2. Requirements

Derived directly from the assignment problem statement:

1. Handle a network with 8,000+ vertices and 20,000+ weighted edges.
2. Support frequent road-weight changes.
3. Remove or temporarily disable blocked roads (and reopen them).
4. Support multiple source-to-destination queries.
5. Answer repeated evacuation queries efficiently (route caching).
6. Keep memory consumption moderate (sparse representation).
7. Avoid repeatedly processing the same intersection.
8. Prioritize routes with minimum travel cost.
9. Identify unreachable evacuation centers.
10. Implement everything in C.
11. Compare Dijkstra with Floyd–Warshall for this network.
12. Analyse complexity, the 200 ms requirement and the behaviour when 30% of the roads become unavailable.

3. Assumptions

- **Safety-adjusted cost.** The assignment asks for the "safest and shortest" route but gives no separate safety formula. The stated engineering assumption is that every non-negative road weight is a *safety-adjusted travel cost* that already folds in distance, congestion, damage and emergency restrictions. Minimizing total weight therefore

minimizes a combined safety-and-time objective, and the authority expresses changing conditions by updating weights.

- **Bidirectional roads.** Roads are bidirectional: one logical road between u and v is stored as two directed edges (u→v and v→u) that are always created, updated, blocked and reopened together. All road counts in this report count each bidirectional road **once**.
- Intersection IDs are arbitrary integers (hence a HashMap, not an array indexed by ID).
- A blocked road keeps its weight and stays in the structure with an `active = 0` flag, so reopening is O(degree) and loses no information.
- Weights fit in a C `long`; the longest possible route cost in the synthetic city (≤ 8,200 hops × 100) is far below `LONG_MAX`.

4. ADT Choice and Graph Representation (justification)

ADT: weighted undirected graph with mutable edge weights and edge status — the natural model of intersections and roads.

Representation: HashMap-based adjacency list, chosen over an adjacency matrix because the network is sparse:

- Density: E ≈ 21,000 of ~33.6 million possible pairs (~0.06%). A matrix would waste > 99.9% of its cells; with 4-byte entries it costs ~269 MB for V = 8,200, versus O(V + E) ≈ a few MB for the list.
- Dijkstra only needs "iterate over neighbours of u", which the adjacency list provides in O(degree(u)); a matrix row costs O(V).
- The HashMap (intersection ID → `Vertex*`) gives O(1) average lookup for arbitrary, non-contiguous IDs, which arrays cannot do.

Supporting ADTs, each mapped to a requirement:

Requirement	ADT	Where
O(1) vertex lookup by ID	HashMap	`src/hashmap.c`, used by `src/graph.c`
Never process an intersection twice	HashSet	`src/hashset.c`, used inside `dijkstra_core`
Pick the cheapest frontier vertex fast	Min-heap priority queue	`src/minheap.c`

Requirement	ADT	Where
Minimum-cost route + unreachable detection	Dijkstra	`src/dijkstra.c`
Repeated queries answered fast	Route cache (hash table + version stamps)	`src/route_cache.c`

5. C Structure Design

All structures are in the corresponding headers under `src/`.

```
typedef struct Edge {        /* one directed half of a road */
    int dest_id;             /* destination intersection    */
    long weight;             /* safety-adjusted cost, >= 0  */
    int active;              /* 1 = open, 0 = blocked       */
    struct Edge *next;       /* next edge in adjacency list */
} Edge;

typedef struct Vertex {      /* one intersection            */
    int id;                  /* external ID (any int)       */
    int index;               /* dense 0..n-1 array index    */
    Edge *edges;             /* adjacency list head         */
    int degree;
} Vertex;

typedef struct Graph {
    HashMap *vertices;       /* ID -> Vertex*               */
    Vertex **by_index;       /* dense index -> Vertex*      */
    int num_vertices;
    int cap_vertices;
    long num_logical_roads;  /* each road counted once      */
    unsigned long version;   /* bumped on routing changes   */
} Graph;

typedef struct HashMapEntry {
    int key; void *value; struct HashMapEntry *next;
} HashMapEntry;
typedef struct HashMap {
    HashMapEntry **buckets; int num_buckets; int size;
} HashMap;

typedef struct HashSetEntry {
    int key; struct HashSetEntry *next;
} HashSetEntry;
typedef struct HashSet {
    HashSetEntry **buckets; int num_buckets; int size;
} HashSet;

typedef struct HeapNode { int vertex_index; long dist; } HeapNode;
```

```
typedef struct MinHeap {
    HeapNode *nodes; int *pos; int size; int capacity;
} MinHeap;

typedef struct Route {
    int src_id, dst_id;
    int *intersection_ids;   /* src..dst in travel order    */
    int num_intersections;
    long total_cost;         /* -1 when unreachable         */
    int reachable;
} Route;

typedef struct RouteCacheEntry {
    int src_id, dst_id;
    unsigned long graph_version;  /* version at compute time */
    Route *route;                 /* owned deep copy          */
    struct RouteCacheEntry *next;
} RouteCacheEntry;

typedef struct RouteCache {
    RouteCacheEntry **buckets; int num_buckets; int size;
    unsigned long hits, misses, stale_evictions;
} RouteCache;
```

The dense `index` on each vertex is a deliberate design point: external IDs are arbitrary, so the HashMap resolves IDs, while Dijkstra's `dist[]`/`prev[]` and the heap's `pos[]` use compact 0..V−1 indices for O(1) array access.

6. HashMap and HashSet Design

Both use **separate chaining** over a power-of-two bucket array with the Knuth multiplicative hash `h(k) = (unsigned)k · 2654435761 mod 2^32`, masked to the bucket count. Both resize (doubling and re-linking every entry) when the load factor exceeds 3/4, keeping average chains short so put/get/contains stay O(1) on average.

- **HashMap** (`int key → void *value`) *is* genuinely used for graph storage*: `Graph.vertices` maps every intersection ID to its `Vertex` record, and every operation — adding roads, updating weights, blocking, and every edge relaxation inside Dijkstra (`graph_get_vertex`) — goes through it.
- **HashSet** (`int` members) is *genuinely used inside Dijkstra*: `dijkstra_core` adds each finalized intersection to the `processed` set and skips any heap node whose vertex is already in it. This is the mechanism that prevents reprocessing and makes cyclic networks safe (functional test 15 verifies it).

7. Min-Heap Design

Array-based binary min-heap keyed by tentative distance, with a `pos[vertex_index]` reverse index so the heap slot of any vertex is found in O(1) — the prerequisite for an O(log V) decrease-key.

- **insert** – place at the end, sift up: O(log V).
- **extract-min** – take root, move last node to the root, **heapify** (sift down) to restore order: O(log V).
- **heapify** – iterative sift-down comparing a node with its two children (`minheap_heapify` in `src/minheap.c`).
- **decrease-key** – locate the vertex via `pos[]`, lower its key, sift up: O(log V). Rejects a key that is not actually smaller.

Every swap updates `pos[]` for both moved nodes, keeping the reverse index exact. Dijkstra calls decrease-key when a shorter path to a vertex already in the heap is found, and insert when the vertex is seen for the first time — the heap is never replaced by an array scan.

8. Dynamic Updates

- `graph_update_road_weight(g, u, v, w)` – validates both endpoints, rejects negative `w`, updates **both** directed halves, bumps `g->version`.
- `graph_set_road_status(g, u, v, active)` – validates, flips the `active` flag on both halves, bumps `g->version`. Blocked edges stay in the adjacency list; Dijkstra skips them with `if (!e->active)`. Reopening restores the road with its stored weight.
- `graph_add_road` – validates endpoints, rejects self-loops, negative weights and duplicates; also bumps the version, since a new road can change best routes.

The single `version` counter is the cache-invalidation mechanism: any routing-relevant mutation increments it, so every cached route knows whether it is still valid (section 11).

9. Dijkstra and Route Reconstruction

`dijkstra_shortest_route` (in `src/dijkstra.c`):

1. Resolve source and destination through the HashMap; return NULL for invalid IDs.
2. Initialise `dist[] = INF`, `prev[] = -1`, `dist[src] = 0`; insert the source into the min-heap.
3. Loop: extract-min; if the vertex is already in the processed HashSet, skip it; otherwise add it to the set. Stop early when the destination is extracted — at that moment its distance is provably optimal (all weights are non-negative).

4. Relax each **active** outgoing edge; on improvement update `dist`/`prev` and insert or decrease-key in the heap.
5. If `dist[dst]` is still INF, the destination is **unreachable**: the function returns a `Route` with `reachable = 0` (a valid answer, not an error).
6. **Route reconstruction:** walk `prev[]` backwards from the destination, count the hops, then fill the ID array in reverse so the route reads source → … → destination, with `total_cost = dist[dst]`.

10. Route Caching

`RouteCache` is a separate hash table keyed by the (src, dst) pair (mixed into one hash). Each entry stores a **deep copy** of the route and the **graph version** at computation time.

- **Lookup**: find the entry; if its stored version equals the graph's current version it is a *hit*. If the versions differ, the entry is stale — it is evicted, `stale_evictions` is incremented, and the lookup reports a *miss*.
- **evac_query** (the query front-end): validates IDs, tries the cache, otherwise runs Dijkstra and stores the result — including *unreachable* results, so repeatedly asking for a cut-off center is also O(1) after the first miss.

This version-stamp scheme makes invalidation O(1) per mutation (just an integer increment), at the cost of invalidating all cached routes after any change — the correct trade-off during a disaster, when any road change can affect arbitrarily many routes.

11. Error Handling

Condition	Behaviour	Where verified
Invalid source/destination in a query	`dijkstra_shortest_route`/`evac_query` return NULL	tests 6, 7
Invalid endpoint in add/update/block	`GRAPH_ERR_INVALID_VERTEX`	demo step 10
Duplicate road	`GRAPH_ERR_DUPLICATE_ROAD`, graph unchanged	test 8
Duplicate intersection	`GRAPH_ERR_DUPLICATE_VERTEX`	`graph_add_intersection`

Condition	Behaviour	Where verified
Negative weight (add or update)	`GRAPH_ERR_NEGATIVE_WEIGHT`, rejected	test 9
Self-loop road	`GRAPH_ERR_SELF_LOOP`	`graph_add_road`
Update/block a road that does not exist	`GRAPH_ERR_NO_SUCH_ROAD`	`graph.c`
Unreachable destination	`Route.reachable = 0`, cost −1, reported	tests 10, 16
Cycles	HashSet guarantees each vertex processed once	test 15
Allocation failure	every allocation checked; partial work rolled back (e.g. a half-added road)	all modules
Memory deallocation	`graph_free`, `route_free`, `route_cache_free`, `hashmap_free`, `hashset_free`, `minheap_free`; every test/demo/benchmark path frees what it creates	all modules

12. Pseudocode

Original pseudocode matching the implementation (file references in parentheses).

12.1 Graph insertion (`graph.c`)

```
AddIntersection(G, id):
    if HashMapContains(G.vertices, id): return DUPLICATE_VERTEX
    v = new Vertex(id, index = G.num_vertices, edges = empty)
    HashMapPut(G.vertices, id, v); append v to G.by_index
    return OK

AddRoad(G, u, v, w):                  # bidirectional
    if u or v unknown: return INVALID_VERTEX
    if u == v: return SELF_LOOP
    if w < 0:  return NEGATIVE_WEIGHT
    if edge u->v exists: return DUPLICATE_ROAD
    prepend Edge(v, w, active=1) to u.edges
    prepend Edge(u, w, active=1) to v.edges
```

```
    G.num_logical_roads += 1; G.version += 1
    return OK
```

12.2 Road weight update (`graph.c`)

```
UpdateRoadWeight(G, u, v, w):
    if u or v unknown: return INVALID_VERTEX
    if w < 0:          return NEGATIVE_WEIGHT
    e1 = FindEdge(u, v); e2 = FindEdge(v, u)
    if e1 or e2 missing: return NO_SUCH_ROAD
    e1.weight = w; e2.weight = w
    G.version += 1                    # invalidates cached routes
    return OK
```

12.3 Road blocking / reopening (`graph.c`)

```
SetRoadStatus(G, u, v, active):
    if u or v unknown: return INVALID_VERTEX
    e1 = FindEdge(u, v); e2 = FindEdge(v, u)
    if e1 or e2 missing: return NO_SUCH_ROAD
    e1.active = active; e2.active = active
    G.version += 1
    return OK
```

12.4 HashMap operations (`hashmap.c`)

```
Hash(key): return (unsigned(key) * 2654435761) mod 2^32

Put(M, key, value):
    if M.size > 3/4 * M.num_buckets: Resize(M)   # double + re-link
    b = Hash(key) AND (M.num_buckets - 1)
    for entry in chain M.buckets[b]:
        if entry.key == key: entry.value = value; return REPLACED
    prepend new entry(key, value) to M.buckets[b]; M.size += 1
    return INSERTED

Get(M, key):
    b = Hash(key) AND (M.num_buckets - 1)
    for entry in chain M.buckets[b]:
        if entry.key == key: return entry.value
    return NOT_FOUND
```

12.5 HashSet insertion and lookup (`hashset.c`)

```
Add(S, key):
    resize as in HashMap when load > 3/4
    b = Hash(key) AND (S.num_buckets - 1)
    if key already in chain S.buckets[b]: return ALREADY_PRESENT
```

```
    prepend key to S.buckets[b]; S.size += 1; return ADDED

Contains(S, key):
    b = Hash(key) AND (S.num_buckets - 1)
    return key in chain S.buckets[b]
```

12.6 Min-heap operations (`minheap.c`)

```
Insert(H, v, d):                        # O(log n)
    H.nodes[H.size] = (v, d); H.pos[v] = H.size; H.size += 1
    SiftUp(H, H.size - 1)

ExtractMin(H):                          # O(log n)
    min = H.nodes[0]; H.pos[min.v] = ABSENT; H.size -= 1
    if H.size > 0:
        H.nodes[0] = H.nodes[H.size]; H.pos[H.nodes[0].v] = 0
        Heapify(H, 0)
    return min

Heapify(H, i):                          # sift-down
    loop:
        smallest = the index with the smallest dist among
                   i, left(i), right(i) that lie inside the heap
        if smallest == i: stop
        Swap(H, i, smallest)            # swap also fixes pos[]
        i = smallest

DecreaseKey(H, v, d):                   # O(log n)
    i = H.pos[v]
    if i == ABSENT or d >= H.nodes[i].dist: return ERROR
    H.nodes[i].dist = d; SiftUp(H, i)
```

12.7 Dijkstra (`dijkstra.c`)

```
Dijkstra(G, src, dst):        # dst = NONE: settle all vertices
    for each vertex i: dist[i] = INF; prev[i] = NONE
    dist[src] = 0
    heap = MinHeapCreate(V); Insert(heap, src, 0)
    processed = HashSetCreate()
    while heap not empty:
        (u, d) = ExtractMin(heap)
        if HashSetContains(processed, u): continue  # finalized
        HashSetAdd(processed, u)
        if u == dst: break                          # early exit
        for each edge e in u.edges:
            if not e.active: continue     # blocked road
            v = HashMapGet(G.vertices, e.dest).index
            if HashSetContains(processed, v): continue
            if dist[u] + e.weight < dist[v]:
                dist[v] = dist[u] + e.weight; prev[v] = u
                if v in heap: DecreaseKey(heap, v, dist[v])
                else:         Insert(heap, v, dist[v])
```

```
    return dist, prev
```

12.8 Route reconstruction (`dijkstra.c`)

```
ReconstructRoute(G, src, dst, dist, prev):
    if dist[dst] == INF: return Route(reachable = false)
    hops = 1; at = dst
    while prev[at] != NONE: hops += 1; at = prev[at]
    ids = array of length hops; at = dst
    for i = hops-1 down to 0: ids[i] = ID(at); at = prev[at]
    return Route(ids, total_cost = dist[dst], reachable = true)
```

12.9 Route-cache lookup and insertion (`route_cache.c`)

```
CacheLookup(C, G, src, dst):
    b = PairHash(src, dst) AND (C.num_buckets - 1)
    for entry in chain C.buckets[b] where entry matches (src, dst):
        if entry.graph_version == G.version:
            C.hits += 1; return entry.route          # HIT
        unlink and free entry                        # stale
        C.stale_evictions += 1; break
    C.misses += 1; return NOT_FOUND                  # MISS

CacheStore(C, G, route):
    entry = (src, dst, G.version, DeepCopy(route))
    prepend entry to its bucket chain
```

12.10 Cache invalidation

```
# There is no invalidation pass. Every mutation does:
#     G.version += 1
# and CacheLookup treats any entry whose stored version
# differs from G.version as invalid, evicting it lazily
# on the next lookup.
```

12.11 Multiple-query processing (`route_cache.c: evac_query`)

```
EvacQuery(G, C, src, dst):
    if src or dst not in G.vertices: return INVALID
    r = CacheLookup(C, G, src, dst)
    if r != NOT_FOUND: return r                      # cache HIT
    r = Dijkstra + ReconstructRoute                  # cache MISS
    CacheStore(C, G, r)      # unreachable results are cached too
    return r
# The dispatcher just calls EvacQuery for each request.
```

13. Functional Test Results

Sixteen tests in `tests/test_runner.c`; full expected/actual/PASS-FAIL transcript in `results/test_results.txt`. Executed result: **16/16 PASS** (exit code 0).

#	Test	Result
1	Known graph, hand-verified shortest route (1→4 = cost 8 via 1-3-2-4)	PASS
2	Route reconstruction (1→5 = cost 11, 5 intersections)	PASS
3	Weight update 3-2: 2→10 changes best route to 1-2-4 (cost 9)	PASS
4	Blocking 2-4 forces alternative 1-3-4 (cost 13)	PASS
5	Reopening 2-4 restores route 1-2-4 (cost 9)	PASS
6	Invalid source rejected	PASS
7	Invalid destination rejected	PASS
8	Duplicate road rejected	PASS
9	Negative weight rejected (add and update)	PASS
10	Unreachable destination reported	PASS
11	Multiple queries (costs 9, 12, 8)	PASS
12	Cache miss on first query	PASS
13	Cache hit on repeated query	PASS
14	Cache invalidation after weight update	PASS

#	Test	Result
15	Cyclic graph, each vertex processed at most once	PASS
16	Isolated vertex (unreachable both ways, reachable count 1)	PASS

14. Large-Network Benchmark

Deterministic synthetic city, generated with the xorshift32 PRNG and fixed seed **20260831** (identical on every run and platform):

- **8,200 intersections** (> 8,000 required);
- **21,000 logical bidirectional roads** (> 20,000 required), stored as 42,000 directed edges but counted once per road;
- a random spanning tree guarantees initial connectivity, then extra random roads (duplicates and self-loops rejected and retried);
- weights uniform in 1..100 (non-negative);
- 10 deterministic source-to-destination queries.

Method: standard C `clock()` (~1 ms resolution). Each uncached figure is the average of 3 back-to-back Dijkstra runs; each cached figure averages 20,000 cache lookups; 5 passes give min/avg/max per query. Graph generation and file writing are **outside** every timed region. Raw data: `results/benchmark_results.csv`.

Metric (per query, across 10 queries × 5 passes)	Uncached (full Dijkstra)	Cached (route cache hit)
Minimum	0.000 ms (below clock resolution)	0.000000 ms
Average of per-query averages	≈ 2.95 ms	≈ 0.00003 ms
Maximum	5.667 ms	0.0004 ms

All 10 destinations were reachable; costs 154–265, routes 5–13 intersections. Cache statistics for the cached-timing loops: 1,000,000 hits, 10 misses (the initial priming), 0 stale evictions.

15. Compilation Record and Environment

- **Machine:** Intel Core Ultra 7 255H (16 cores), 31.4 GB RAM, Windows 11 Home Single Language (build 26200).
- **Compiler available on the machine:** GCC 3.4.2 (mingw-special, Dev-C++). It predates `-std=c11`/`-Wpedantic`, so the closest strict flag set it supports was used. The code is standard C that is valid under both C99 and C11; the Makefile keeps the preferred `gcc -std=c11 -O2 -Wall -Wextra -Wpedantic` for modern compilers.
- **Actual commands and results:**

```
C:\Dev-Cpp\bin\gcc.exe -std=c99 -O2 -Wall -Wextra -pedantic
                       -c <each of the 8 .c files>
  -> every translation unit compiled with ZERO warnings, exit 0
```

- **Linking caveat (honest record):** on this Windows 11 machine the GCC 3.4.2 link driver crashed ("Internal error: Aborted (program collect2)") for any link, even a single trivial object. The object files themselves are fine; linking was completed by invoking the toolchain's own `ld` directly with the exact library list collect2 would have passed (`-lmingw32 -lgcc -lmoldname -lmingwex -lmsvcrt -luser32 -lkernel32 -ladvapi32 -lshell32` plus `crt2.o`, `crtbegin.o`, `crtend.o`). Both `bin/evacsim.exe` and `bin/test_runner.exe` link and run correctly. On a modern GCC, `make` (or `build.bat`) performs the whole build in one step.

16. The 200 ms Requirement

Met on this machine, with the configuration stated here. The worst measured uncached query on the full network (8,200 intersections, 21,000 roads) took **5.667 ms**, and the average was **≈ 2.95 ms** — roughly 35× under the 200 ms budget; cache hits are ~5 orders of magnitude faster still. Configuration: the machine above, GCC 3.4.2 at `-O2`, seed-20260831 synthetic dataset, `clock()` timing.

Honest caveats: `clock()` has ~1 ms resolution (hence batched timing); a slower CPU, a denser network, or much longer routes would raise the numbers. The complexity analysis (section 19) says the margin is structural, not accidental: $O((V+E) \log V) \approx (8{,}200 + 42{,}000) \times 13 \approx 6.5 \times 10^5$ heap-weighted operations per uncached query, far from any 200 ms cliff at this scale — but the claim made here is only about the measured configuration.

17. 30% Road-Failure Analysis

Exactly 6,300 of the 21,000 logical roads (30%) were blocked, selected by a Fisher–Yates shuffle seeded with 20260831 ^ 0xC0FFEE — fully deterministic and reproducible. The same 10 queries were rerun. Full data: `results/road_failure_analysis.txt`. Observations (actual run):

- **Reachability:** all 10 query destinations remained reachable. Global connectivity from intersection 4546 dropped from 8,200/8,200 to **8,026/8,200** — 174 intersections (~2.1%) were cut off entirely. With ~70% of ~2.6 average logical degree remaining, the giant component survives, but its fringes fray.
- **Route costs rose for every query:** from +3 up to +175 (e.g. query 7: 238 → 413, +74%). Median increase ≈ +100.
- **Route lengths mostly grew** (e.g. 5 → 11 and 10 → 19 intersections) as detours replaced blocked segments; one query found a *shorter-hop* but similar-cost alternative (12 → 7 hops, +3 cost).
- **Runtime stayed in the same few-millisecond band** (per-query averages 0.0–7.7 ms, vs 0.0–6.0 ms before). Dijkstra still scans blocked edges' `active` flags, so work does not drop proportionally; all measurements remain far under 200 ms.
- **Cache invalidation worked as designed:** blocking bumped the graph version 6,300 times; on the rerun every one of the 10 cached routes was detected as stale and evicted (stale_evictions = 10), then recomputed against the damaged network.
- **Evacuation-reliability reading:** a city that loses 30% of its roads still routes most evacuees, but ~2% of intersections lose access entirely and surviving routes become substantially more expensive — a strong argument for identifying and hardening the bridge-like roads whose loss isolates neighbourhoods.

18. Dijkstra versus Floyd–Warshall (for this network)

Aspect	**Dijkstra (binary heap + adjacency list)**	**Floyd–Warshall**
Problem solved	Single-source (with early exit: single-pair)	All-pairs
Time	$O((V + E) \log V)$ per query ≈ 6.5×10^5 weighted ops	$O(V^3) \approx 5.5\times10^{11}$ ops for V = 8,200
Space	O(V + E) graph + O(V) working	$O(V^2)$ distance matrix ≈ 67.2 M cells (~269 MB with 4-byte entries; ~537 MB to also store successors for path reconstruction)

Aspect	Dijkstra (binary heap + adjacency list)	Floyd–Warshall
Sparse-graph fit	Excellent: cost scales with E, and $E \approx 21{,}000 \ll V^2$	Poor: cost is V^3 regardless of sparsity
Dynamic updates	Nothing to rebuild; the next query just reads current weights/flags	Any single change can invalidate the whole matrix; recomputing costs $O(V^3)$ again
Repeated queries	Route cache gives O(1) average repeats; invalidation is one integer bump	O(1) lookups *if* the matrix is affordable and static — neither holds here
Negative edges	Not supported (not needed: weights are non-negative by design)	Supported (no negative cycles)
Verdict for 8,200 V / 21,000 E dynamic network	**Suitable — chosen**	**Unsuitable at full scale**

Floyd–Warshall is unsuitable here for three independent reasons: the $O(V^2)$ matrix alone violates the moderate-memory constraint; the $O(V^3)$ computation is ~10^5× more work than answering every benchmark query individually; and the network changes constantly, so the expensive all-pairs answer would be stale almost immediately. Floyd–Warshall was **not** run on the full network; this comparison is analytical, using the standard complexities above. (It remains the right tool for small, dense, static subproblems, e.g. a distance table among a few dozen evacuation centers.)

19. Complexity of the Complete System

Let V = intersections, E = logical roads, deg = E/V, L = route length, C = cached routes.

Operation	Time	Space
HashMap put/get/contains	O(1) average, O(n) worst	O(V) total

Operation	Time	Space
HashSet add/contains	O(1) average	O(V) per search
Add intersection	O(1) average	O(1)
Add road	O(deg) duplicate check + O(1)	O(1)
Update weight / block / reopen	O(deg) edge find ×2	O(1)
Heap insert / extract-min / decrease-key / heapify	O(log V)	O(V) total
Dijkstra query (uncached)	O((V + E) log V)	O(V) (dist, prev, heap, set)
Route reconstruction	O(L)	O(L)
Cache lookup / store	O(1) average	O(C·$\bar{L}$) total
Cache invalidation (per mutation)	O(1) (version bump; stale entries evicted lazily)	—
Graph storage	—	O(V + E)

Whole-system memory is O(V + E + C·$\bar{L}$): for the benchmark city, tens of thousands of small heap blocks — a few megabytes — satisfying the moderate-memory constraint.

20. Limitations and Improvements

Limitations: single scalar cost (no separate safety objective or multi-criteria optimization); global cache invalidation (any change evicts everything — correct but coarse); single-threaded; `clock()` resolution ~1 ms; synthetic grid-less topology rather than real map data; no persistence or network interface; blocked edges still cost a flag check during scans.

Improvements worth pursuing: A* with a landmark or geographic heuristic for faster point-to-point queries; bidirectional Dijkstra; finer-grained invalidation (e.g. only evict routes whose region changed); a contraction-hierarchy preprocessing layer for near-instant repeated queries; multi-criteria (cost, capacity, risk) routing; batching evacuee flows with capacity constraints (a min-cost-flow extension); and reading real road data (e.g. OpenStreetMap extracts).

21. Technical Reflection

The central design lesson was that **the data structures, not the algorithm, decide whether Dijkstra is fast**: the same algorithm backed by an array scan is $O(V^2)$, and with a binary heap plus adjacency list it becomes $O((V+E) \log V)$ — the difference between comfortable milliseconds and a risky budget at 8,200 vertices. The `pos[]` reverse index inside the heap was the subtlest part: decrease-key is only O(log V) because every swap maintains it, and an early version that forgot to update it on extract-min produced wrong routes immediately — caught by test 1's hand-verified route, which reinforced the value of manually checkable test graphs. The second lesson was about **change**: the version-stamp cache made invalidation trivial and impossible to forget, whereas eagerly walking the cache on every road update would have been slower and easier to get wrong. Keeping blocked roads in the list (flagged inactive) instead of deleting them made blocking/reopening symmetric and loss-free. Finally, testing the failure path (unreachable centers, isolated vertices, invalid IDs) mattered as much as the happy path — in an evacuation system, a wrong "no route" is as dangerous as a wrong route.

22. SDG Relevance

- **SDG 9 – Industry, Innovation and Infrastructure:** the project is resilient-infrastructure software: it models a city's road infrastructure, quantifies how it degrades (the 30% failure study), and shows how algorithmic innovation (heap-optimized routing, caching) keeps critical services running on modest hardware.
- **SDG 11 – Sustainable Cities and Communities:** directly supports target 11.5 (reduce deaths and losses from disasters) and 11.b (disaster-resilience planning): minimum-cost evacuation routes, identification of unreachable neighbourhoods, and rapid re-routing when roads close are exactly the capabilities city disaster authorities need.
- **SDG 13 – Climate Action:** climate change increases the frequency of floods, storms and fires that this system responds to; adaptive evacuation routing is a concrete climate-adaptation measure, and the failure analysis quantifies how much redundancy a road network needs as extreme events intensify.

23. Conclusion

The delivered prototype meets every stated constraint: a HashMap-based adjacency list stores an 8,200-intersection / 21,000-road city in O(V+E) memory; a min-heap with a working decrease-key drives Dijkstra to answer minimum-cost queries in ≈ 3 ms (worst measured 5.7 ms, budget 200 ms); a HashSet guarantees each intersection is processed once even in cyclic networks; dynamic weight updates and road blocking/reopening take effect immediately and invalidate the version-stamped route cache correctly; unreachable centers are detected and reported; and all 16 functional tests pass. The 30% failure experiment shows the design degrades gracefully — costs rise and ~2% of intersections become isolated, but routing stays fast and correct — which is precisely the behaviour an emergency evacuation system must have.

24. References

1. E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
2. R. W. Floyd, "Algorithm 97: Shortest Path," *Communications of the ACM*, vol. 5, no. 6, p. 345, 1962.
3. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009 — ch. 6 (heaps), ch. 11 (hash tables), ch. 24–25 (shortest paths).
4. D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed., Addison-Wesley, 1998 — §6.4 (hashing).
5. G. Marsaglia, "Xorshift RNGs," *Journal of Statistical Software*, vol. 8, no. 14, 2003.
6. United Nations, *Transforming Our World: the 2030 Agenda for Sustainable Development* (SDGs 9, 11, 13), 2015. https://sdgs.un.org/2030agenda
7. ISO/IEC 9899:2011, *Information technology — Programming languages — C* (C11).

25. Rubric-Compliance Checklist

See `report/rubric_checklist.md` for the detailed mapping. Summary:

Rubric area (marks)	**Where satisfied**
Problem understanding & ADT selection (15)	Report §1–§4; `src/graph.h` design notes
Graph, HashMap & HashSet design (20)	§5–§6; `src/graph.c`, `src/hashmap.c`, `src/hashset.c`; tests 8–10, 15

Rubric area (marks)	Where satisfied
Min-heap & priority-queue integration (20)	§7, §12.6; `src/minheap.c`; used in `src/dijkstra.c`
Dijkstra & route reconstruction (20)	§9, §12.7–12.8; `src/dijkstra.c`; tests 1–5, 15
Performance, testing & complexity (15)	§13–§19; `results/*` (all three files)
Reflection & SDG relevance (10)	§21–§22

GitHub Repository: [Insert repository link after upload] (A local Git repository with the full history is prepared in the project folder; the link above is to be filled in once the repository is published.)